Templates

A MINIMAL APPLICATION WITH THE COMPLETE FRAMEWORK STRUCTURE

START, EXTEND, TEST AND OPERATE

Developer and Operator Guide

Identity, a visual SmartList and ordinary chat, using the same development and deployment foundations as MyScoutee.

DOCUMENT ID	DOCUMENT VERSION
T-START-001	1.0.0
SOFTWARE VERSION	**RELEASED**
1.0.0	2026-10-12
STATUS	
Implementation qualification in progress	

Document control

This HTML file is the canonical manual. Generate the single stable PDF path with `node guides/manuals/build-manuals.mjs T-START-001`. Git records revisions; PDF filenames do not contain a changing version. The document is not evidence of an installed or published release.

The numbered plan in `guides/plans/MSC-122_TEMPLATE_IMPLEMENTATION_PLAN.md` and the QA evidence identify passed, failed and pending gates. Local VMware installation is deferred to the next attended session. Netcup is outside the authorized work: do not connect to or change it.

Contents

Framework and domain selection

Checkout and frontend ownership

Local and HTTP development

Automated qualification

KVM and monitoring

Data and account lifecycle

Build another application

Installed operation and releases

1. Framework and domain selection

The landing page shows Templates, login and the version badge. Members have a visual sample-card SmartList and ordinary chat. The avatar menu retains the small profile and image editor, API access and minimal settings. Notifications, guides, help documents and consent remain available. The administrator shell has no product navigation; administrator API access remains. Operator configuration, email, TLS and update surfaces are retained.

Chat includes message editing, deletion, replies, reactions, pins, read state, reconnect and two-party audio/video calls. It excludes MyScoutee invitations, reports, feedback and support-case management. Calls use encrypted WebRTC media and STUN discovery. A local fake-camera test does not prove physical devices, public-network traversal or a TURN relay.

There is no event, community/group, matchmaking, payment or virtual-company Registry implementation. Necessary account and chat model fields retain their original meaning. Optional product fields are omitted from JSON rather than recreated as empty product domains. Localization and geographic location are both retained.

2. Checkout and frontend ownership

The backend is the parent repository. `frontend` remains its own local Git submodule. Its exact source commit is carried in `tools/sources/frontend.bundle`, a standard Git transport archive. Run `tools/bootstrap-submodules.sh` after cloning the parent; it verifies the bundle against the gitlink and initializes the frontend locally. The frontend's `components` submodule is fetched at its recorded commit from `https://github.com/fssrepository/myscoutee-components.git`.

Commit frontend changes first, run `tools/sync-local-frontend.sh`, and commit the updated gitlink and bundle together in the parent. The bootstrap refuses a dirty frontend. To redirect the frontend later, first publish its recorded commit to the new repository, then run `git submodule set-url frontend <repository-url>` and `git submodule sync -- frontend`. Commit that URL and remove the unused bundle; the same bootstrap command will use ordinary Git.

Use the checked-in Gradle wrapper and frontend lockfile. The development environment uses Java 25, Node 24, Docker Compose and a supported Chrome/Chromium. Install frontend dependencies with `npm ci` in `frontend` after initializing components. Never copy local credentials, generated databases or build caches into Git.

3. Local and HTTP development

Browser-local mode uses IndexedDB and the selected four-person seed. In `frontend`, run `npm run start:local` and open `http://127.0.0.1:4200`. It is a UI/data-adapter environment, not a backend or live WebRTC deployment.

HTTP development uses the versioned `server/docker-compose-dev.yml` and `server/docker-dev.env`. Its existing backend hot-deploy and frontend synchronization process own compilation and asset delivery. Inspect that process and its logs before starting a build. Do not start another watcher or compile the same target alongside it.

The frontend keeps distinct local, dev, E2E, production and GitHub Pages configurations. Production output belongs to `frontend/prod`; Pages output belongs to `frontend/docs`. Frontend synchronization, runtime availability and build stamping remain part of the workflow. Preserve relative assets and the configured base path when changing the application name.

Stop the active dev stack before starting KVM, E2E or another application stack. A monolith and the independent-service lab are alternate deployments, not parallel development servers.

4. Automated qualification

`tools/run-backend-junit.sh standard` runs the selected Java unit tests with isolated outputs. Use its `--tests` filter for a focused change; embedded Mongo checks use the separate `embedded` group. Frontend tests use `npm test -- --watch=false`. Do not rerun every suite after each small edit.

`e2e/README.md` defines the disposable E2E composition, certificate trust, fixture variables and serial runners. Run `node e2e/functional/backend/run.mjs` for the retained API contracts and `node e2e/functional/ui/run.mjs --fail-fast` for browser flows. `--case` selects one matching flow. Each fixture owns its actors and removes them afterward; it must not reset production data or another user's sessions.

Record actual build revisions, image IDs, environment and fixture scope. Browser-local success does not qualify HTTP, and a healthy container does not prove a working user flow. Compare CPU and latency using identical workloads and limits after a coherent change batch. Stress-tool selection and a repeatable template baseline remain explicit plan items.

5. KVM and monitoring

The template owns a separate VM named `myscoutee-template`, with four vCPUs and 6 GiB RAM. Its private keys, disks and Terraform state live outside Git under `~/.local/state/myscoutee-template/microservices`. Host-boot autostart is disabled. The namespace contains four JVM owners:

Owner	Selected domains
identity	Accounts, access, minimal administration, platform, integrations and showcase
communication	Chat, notifications, email and localization
media	Media storage and processing
help	Help, guide and consent documents

The lab includes a two-member Mongo replica set with distinct REAL and DEMO databases, Redis, MinIO, MCP, STUN, ingress and a local release simulator. Two voting Mongo members require both for majority writes; this is not one-node-loss availability.

```
microservices/tools/lab.sh preflight
microservices/tools/lab.sh build
microservices/tools/lab.sh frontend-image
python3 microservices/tools/workloads.py lock-images
```

```
python3 microservices/tools/vm.py start
python3 microservices/tools/workloads.py backends-stop
python3 microservices/tools/workloads.py up
python3 microservices/tools/workloads.py forward
microservices/tools/lab.sh monitoring-up
```

Build support images when their canonical sources change using `lab.sh support-images`, `minio-image` and `release-simulator-image`. Startup imports locked images; it does not silently rebuild. Stop backends before deploying changed writers. `seed-mongo.py` owns Mongo initialization; do not replace it with hand-written import commands.

The application is at `http://localhost:18080`, Grafana at `http://localhost:13000`, and Prometheus at `http://localhost:19090`. The loopback-only Grafana login is `lab / template-local-only`. Use `microservices/tools/open-terminals.sh` for the live status view or `python3 microservices/tools/vm.py observe --once` for one snapshot. The terminal shows guest/container CPU, RAM, readiness and host monitoring. Closing it does not stop the VM.

```
microservices/tools/lab.sh monitoring-stop
python3 microservices/tools/vm.py stop
```

These commands retain owned data and do not enable autostart. Use the versioned ownership-checked destroy tools only when intentionally discarding a lab; never use global Docker prune as a cleanup substitute.

6. Data and account lifecycle

Canonical Mongo collection/index definitions, selected system catalogs and small demo fixtures are under `server/docker/conf/mongodb`. Production identity records are empty. The operator provisions real credentials and branding after installation. Browser-local seeds use the same selected model meanings.

Account deletion keeps the original grace period, normally 30 days, followed by durable purge retries across the retained owners. Notification outbox delivery, scheduled messages, warning email and integration callback work retain their own state and retry behavior. A removed administrator screen is not a reason to remove necessary lifecycle jobs.

Keep REAL and DEMO boundaries intact in HTTP, internal transport, workers and sockets. Internal actor grants are short-lived and validated per operation. A persistent split-service socket renews through its original account session; message bursts reuse its current grant. Never replace session checks with a caller-supplied user ID.

7. Build another application

Start with the showcase owner and its visual cards. Replace those sample records and presentations with your domain while keeping identity, navigation, popup behavior, SmartList views and chat infrastructure. Add optional JSON fields only when the new application needs them.

Canonical Java implementations remain in `server/projects/profile`. Add a domain's contracts and implementation there, then select its owner in `microservices/assemblies.json`. Cross-owner calls use the existing typed transport and authority protocol. Do not copy the monolith into each

service.

Frontend contracts, services and repositories follow domain folders across base, HTTP and local adapters. Extend the existing component/store/repository owner, preserving observable local/HTTP behavior. Shared visual controls belong to the existing components submodule. `tools/upstream-sources.json` records the source revisions and deliberate adaptations needed for future upgrades.

8. Installed operation and releases

The target is a versioned Debian package and immutable Docker images, signed update metadata, client installation tools and a static Pages build. Packaging, installed backup/restore, update interruption/rollback and actual publication are still under qualification. Use the numbered plan rather than assuming these gates passed because local or KVM tests passed.

DEB qualification will run on a local VMware Debian VM in the next attended session. Keep its evidence separate from KVM. Test fresh installation, restart, backup/restore and data-preserving removal. Do not mark the deferred VMware check as successful.

A release must include the matching signed update metadata beside its DEB, with package and client-tool hashes and a separately trusted public-key fingerprint. Private signing keys and deployment secrets stay outside the repository and package. The updater must reject missing or changed metadata; a signature-verification test alone does not prove an installed update.

Before release acceptance, reproduce a clean checkout with both submodules, build the retained runtime, verify the exact artifacts, and record the remaining external-provider or installed-host limits. The final user smoke check is login plus opening SmartList and chat; detailed automatic qualification remains the project's responsibility.